Sequential Circuit Studio

A browser-based EDA workbench for finite-state machines

From a single state table, derive equations, K-maps, a state diagram, a routed gate-level circuit, an interactive timing simulation and synthesizable Verilog — all client-side, deployed on Cloudflare Pages.

Overview

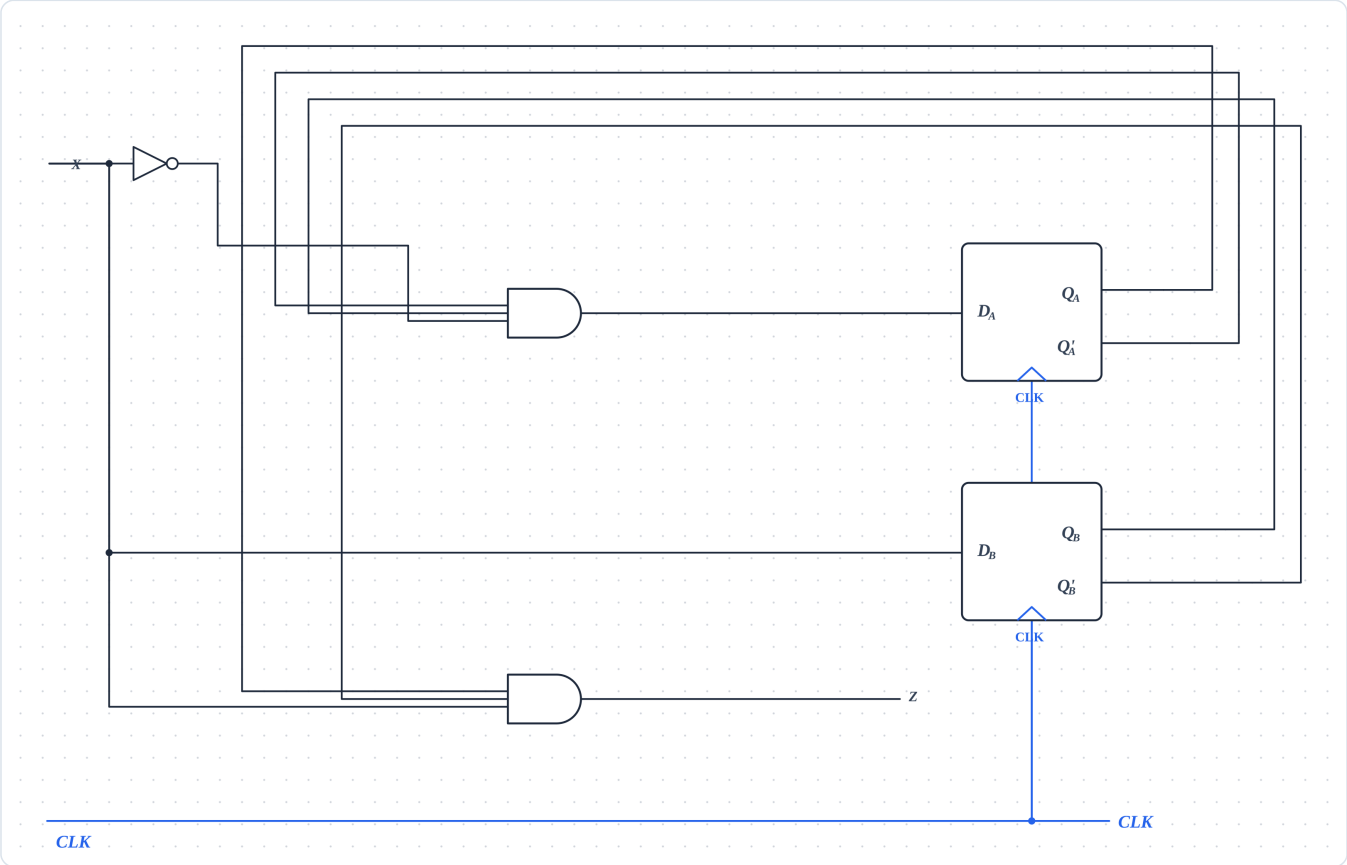
Sequential Circuit Studio turns a finite-state-machine specification into every classic digital-design artifact in real time. You edit one state table on the left; the app derives the next-state and excitation equations, minimizes them, draws the state diagram and a clean gate-level schematic, simulates the timing waveform, and emits behavioral, gate-level and testbench Verilog. Everything runs in the browser with a serverless (Cloudflare) compute fallback — no install, no backend database.

Feature pipeline

- **State table editor** The single source of truth: present state + input -> next state + output, with Moore/Mealy and JK/D/T/SR flip-flop modes.
- **State diagram** An auto-laid-out, curved-edge state graph with non-overlapping transition labels.
- **Excitation table & K-Maps** Per-equation Karnaugh maps with gray-code headers and coloured rings marking each prime-implicant group.
- **Boolean equations** Minimized next-state, excitation and output expressions derived from the table.
- **Circuit diagram** A zone-placed, orthogonally-routed schematic (inputs -> gates -> flip-flops -> outputs) with hover signal-tracing, click-to-inspect parts and a live 0/1/X 'Values' probe.
- **Timing diagram** A clock-driven waveform plus a step-by-step simulation table that checks the FSM against the reset state.
- **Verilog code** Behavioral, gate-level and self-checking testbench Verilog, honoring the configured initial/reset state.
- **Validation** Continuous design lint + verification with a debounced, non-flickering status indicator.

Generated circuit schematic

Authentic output for the built-in example "101 Sequence Detector (Mealy + D)". The layout engine places components in left-to-right zones and routes every net on its own orthogonal lane so different signals never share a wire segment.

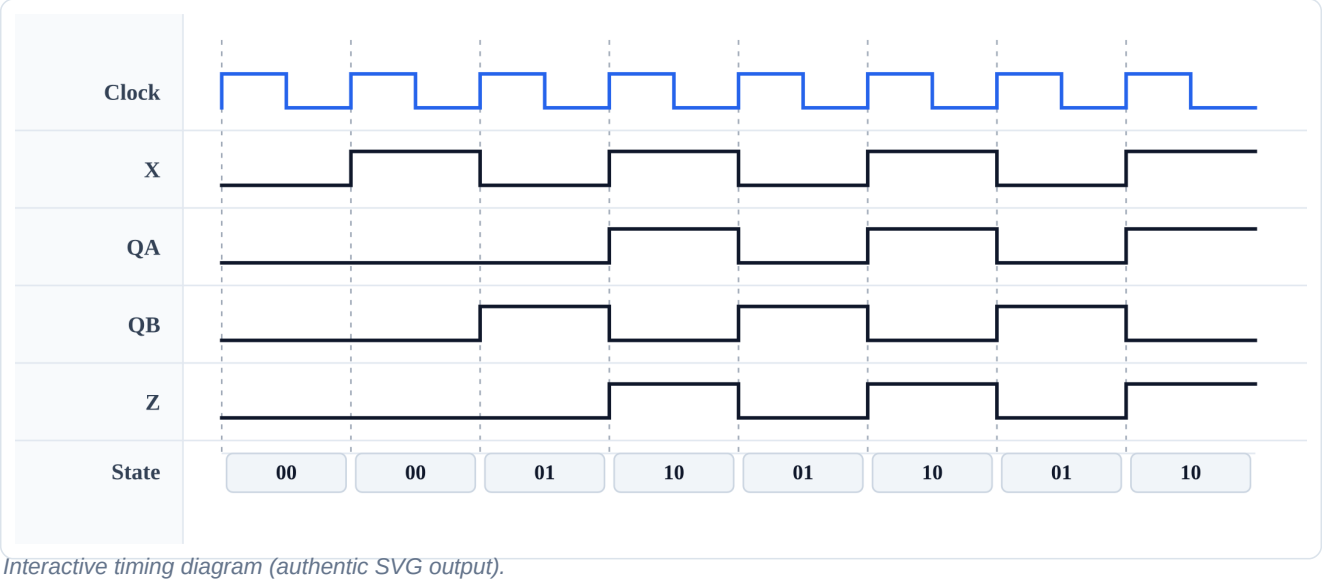


Auto-generated gate-level schematic (the app's own SVG renderer).

Derived equations: D_A = A'BX' D_B = X Z = AB'X

Timing simulation

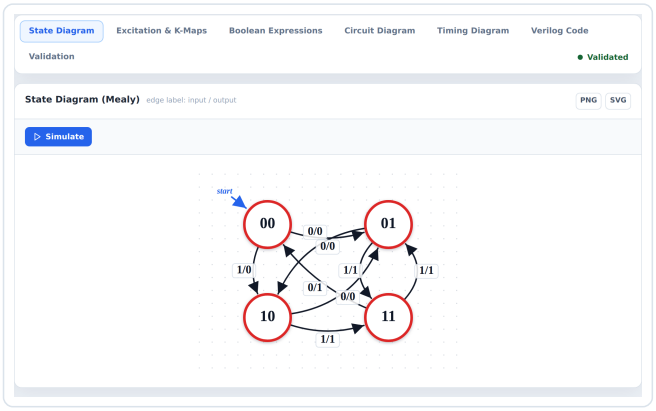
The clock-driven waveform for the same machine, with state annotations underneath each cycle.



Interactive timing diagram (authentic SVG output).

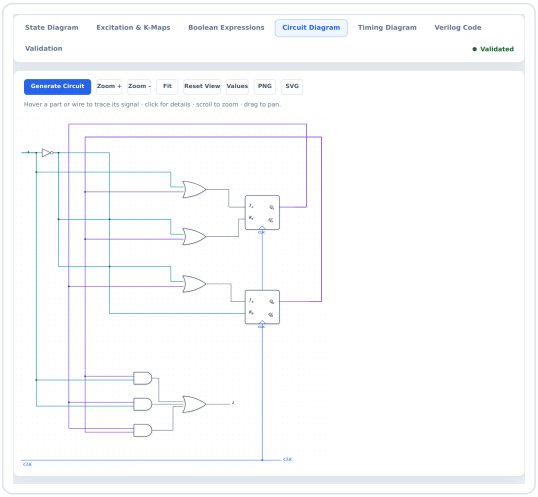
The workbench

Live screenshots of the running app — one tab per design-flow stage, all driven by the same state table.

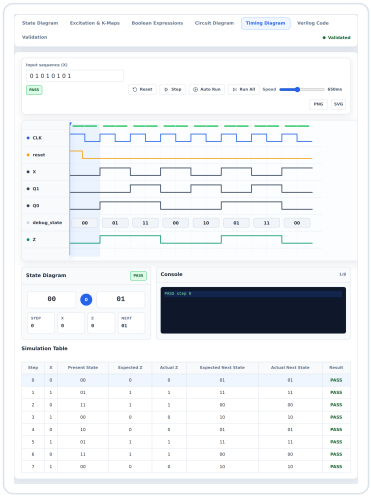


State diagram

Excitation & K-Maps

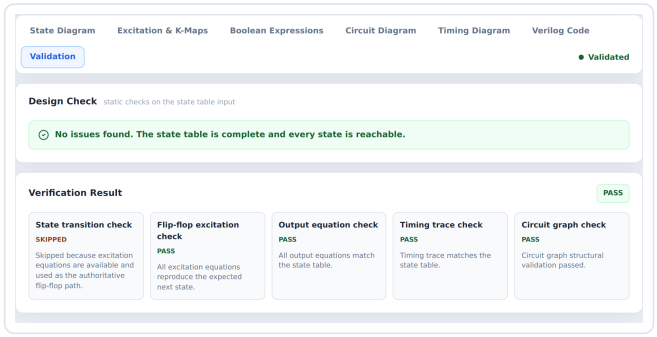


Circuit diagram



Timing simulation

Verilog + testbench



Validation

Architecture

The app is a layered, mostly-pure TypeScript pipeline. UI components are thin; all derivation, layout, routing, simulation and code-generation logic lives in framework-agnostic modules under src/logic, making it testable in isolation (160+ unit tests) and reusable by the Cloudflare Functions backend.

Module layout

- **src/logic/circuit/** The circuit pipeline split into single-responsibility modules: constants, geometry, nets, pins, routing, validation, junctions, render, layout, model and simulate.
- **src/logic (core)** equations, minimizer, kmap, timing, interactiveSimulation, testbenchSimulation, codeGenerator, circuitGraph.
- **src/store** useCircuitStore (Zustand) — workspace state, undo/redo, autosave, share links and a debounced idle/editing/validating/valid/invalid status machine.
- **src/components** React + react-konva views, one per tab, kept presentational.
- **src/api + functions/api** A thin client API with a Cloudflare Functions backend and an in-browser dev fallback, so compute can run locally or serverless.
- **src/lib** verification, design lint and workspace (de)serialization / share-link encoding.

Tech stack

TypeScript · React 18 · Vite · Zustand · Konva / react-konva · Vitest · Cloudflare Pages · Cloudflare Functions · jsPDF · html2canvas

Quality & engineering

- **Deterministic layout & routing** Net-aware orthogonal routing guarantees different nets never share a segment; a canvas-bounds validator keeps everything on-page.
- **Tested** 160+ Vitest cases across logic, layout, routing, simulation, code-gen and the store; production build gated in CI.
- **Stable UX** Debounced validation and a fixed-position status pill eliminate flicker while editing.